

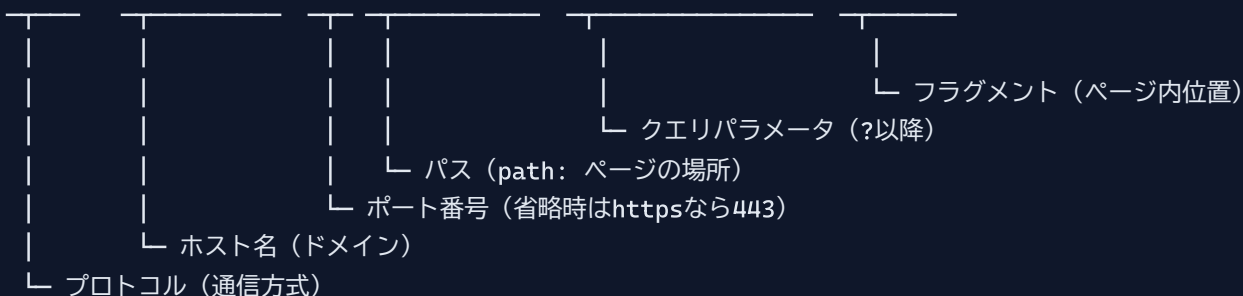
第4章: Next.js の基礎

0. 前提知識: URL・ルーティング・サーバーレンダリング

Next.js を理解するには「URL」「ルーティング」「レンダリング」の3つの基本概念を押さえる必要があります。ここから始めましょう。

0.1 URL の構造

```
https://example.com:443/products/123?sort=price&page=2#reviews
```



Next.js で書く各ページは、このパス（`/products/123` の部分）に対応します。

0.2 ルーティングって何？

「URL を受け取って、どのページを表示するか決める仕組み」が **ルーティング (routing)** です。Next.js の **App Router** では、`app/` フォルダの中のフォルダ構成がそのまま URL になります。

```
app/  
├─ page.tsx          → /          (トップページ)  
├─ about/  
│   └─ page.tsx      → /about  
└─ books/  
    ├─ page.tsx       → /books  
    └─ [id]/  
        └─ page.tsx   → /books/123 ([id] は動的な値)
```

ファイル = ページ: 「`app/about/page.tsx` を作っただけで `/about` というURLでそのページが見られるようになる」というのが Next.js のキモです。第3章までの React だけでは、自分でルーターを書く必要がありました。

0.3 レンダリング3兄弟 (CSR / SSR / SSG)

Webページを「画面に出るHTMLにする」処理を **レンダリング** と呼びます。これがいつ・どこで行われるかで3種類あります。

種類	フルネーム	いつHTMLができる？	どこで？	メリット
CSR	Client Side Rendering	アクセス時	ブラウザ	動的、軽量サーバー
SSR	Server Side Rendering	アクセス時	サーバー	SEO良、初回表示速い
SSG	Static Site Generation	ビルド時に1回	サーバー	最速、安価

本書での使い分け: Next.js の App Router では、`page.tsx` をシンプルに書くとサーバーで実行される（SSR/SSG相当）のがデフォルトです。インタラクティブな部品だけ `"use client"` を付けて CSR にします。

はじめに

この章では、React ベースのフルスタックフレームワーク（Full-stack Framework：フロントエンド=画面もバックエンド=サーバー処理も両方カバーするフレームワーク）である **Next.js**（ネクストジェイエス）の基礎を学びます。

Next.js を使うことで、React 単体では実現が難しい**サーバーサイドレンダリング**（SSR：Server Side Rendering = サーバー側でHTMLを生成してからブラウザに送る方式。ページの初回表示が速くなる）、**静的サイト生成**（SSG：Static Site Generation = ビルド時にHTMLを事前生成する方式。最も高速）、**ファイルベースルーティング**（ファイルを配置するだけでURLとページが自動的に対応する仕組み）などを簡単に実装できます。

なぜ React だけではなく Next.js を使うのか

React は「UIの部品を作るライブラリ」であり、それ以外の機能（ページのURL管理、サーバーとの通信、SEO対策など）は自分で用意する必要があります。Next.js はこれらを**最初から備えている**ため、開発者はアプリのロジックに集中できます。

機能	React のみ	Next.js
ページのURL管理（ルーティング）	別途ライブラリ（react-router等）が必要	ファイルを置くだけで自動対応
サーバーでのデータ取得	自分でAPIサーバーを構築	Server Components で直接取得
SEO（検索エンジン対策）	追加設定が必要	標準で対応
ページ読み込み速度	初回が遅い（CSR）	高速（SSR/SSG）
デプロイ（公開）	別途設定が必要	Vercel で数クリック

この章で学ぶこと: - Next.js の概念と React との関係 - **App Router**（アップルーター：Next.js 13以降の新しいルーティング方式）によるルーティング - **Server Components**（サーバー上で実行されるコンポーネント）と **Client Components**（ブラウザ上で実行されるコンポーネント）の使い分け - レイアウト、ナビゲーション、**データフェッチ**（Data Fetch：サーバーやAPIからデータを取得すること）の基本 - **Server Actions**（サーバーアクション：フォーム送信などのサーバー処理をシンプルに書ける仕組み）によるサーバー処理 - プロジェクト構成の**ベストプラクティス**（Best Practice：最も良いとされるやり方・慣習）

1. Next.js とは

1.1 React との関係

Next.js は **Vercel 社** が開発・メンテナンスしている React ベースのフレームワークです。React はあくまで「UIライブラリ」であり、ルーティングやサーバーサイド処理などは含まれていません。Next.js はその React の上に、Web アプリケーション開発に必要な機能を包括的に提供します。



たとえばで理解する:

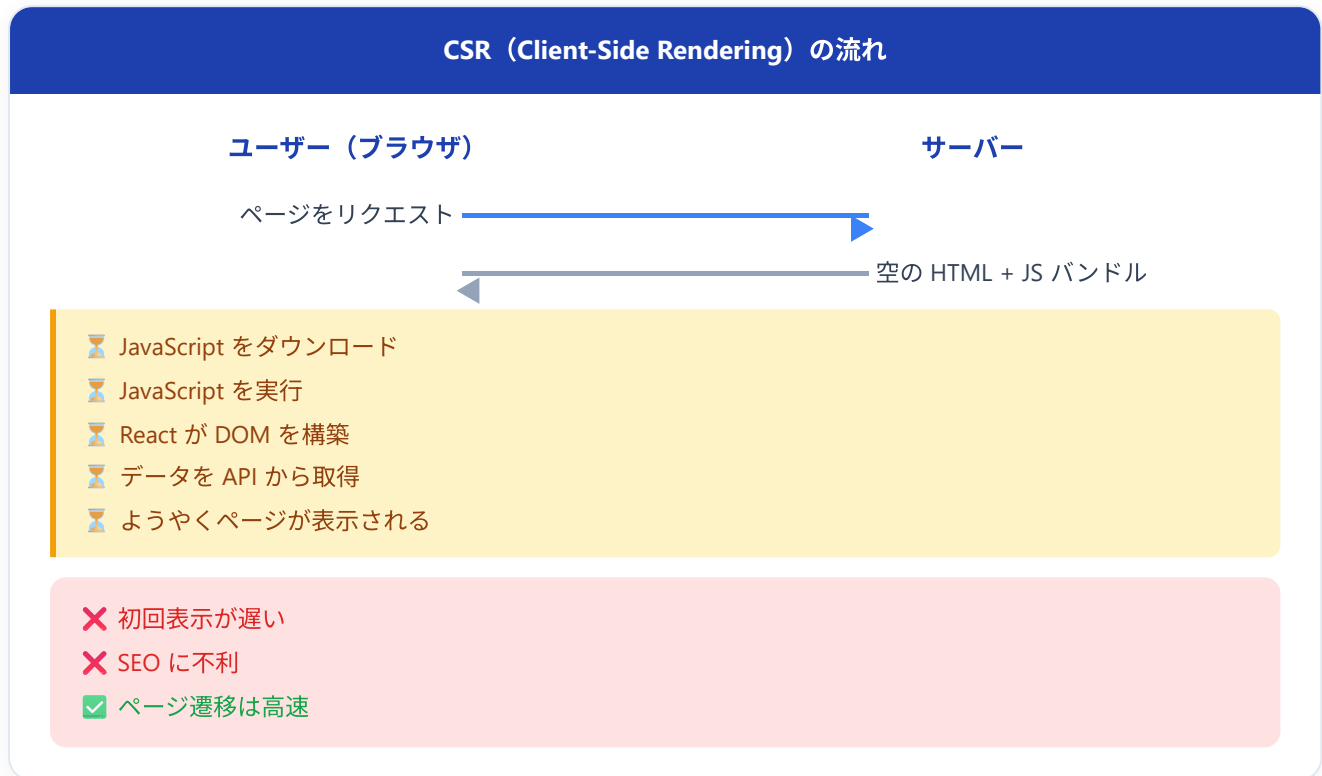
- **React** = エンジン（車を動かす核心部分）
- **Next.js** = 完成した車（エンジンに加え、ハンドル、ブレーキ、ナビなど全部入り）

React 単体でアプリを作ることできますが、ルーティングやデータ取得の仕組みを自分で選定・構築する必要があります。Next.js を使えば、それらが最初から統合されています。

1.2 SSR, SSG, CSR の違い

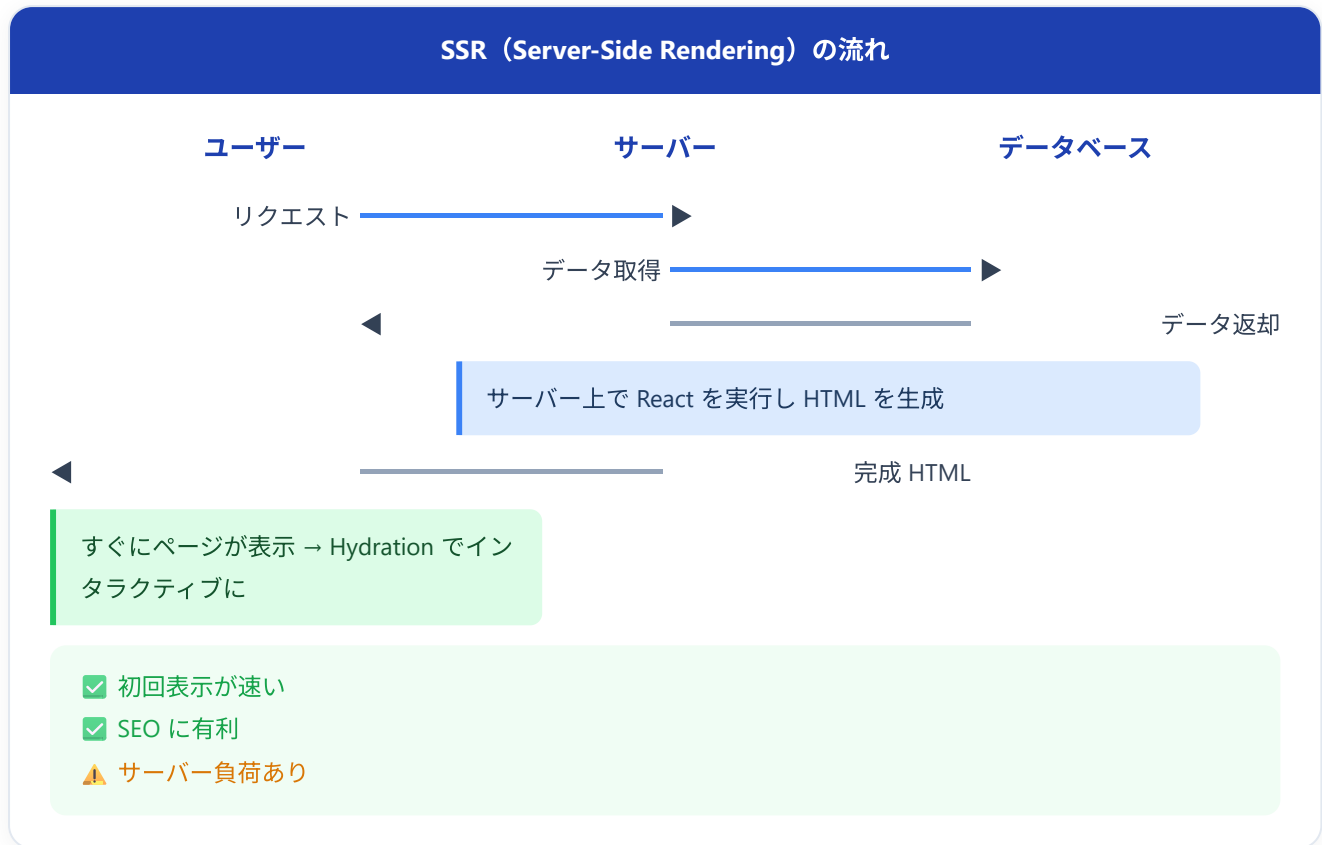
Web ページのレンダリング方法は大きく3つあります。Next.js はこれらすべてに対応しています。

CSR (Client-Side Rendering) - クライアントサイドレンダリング



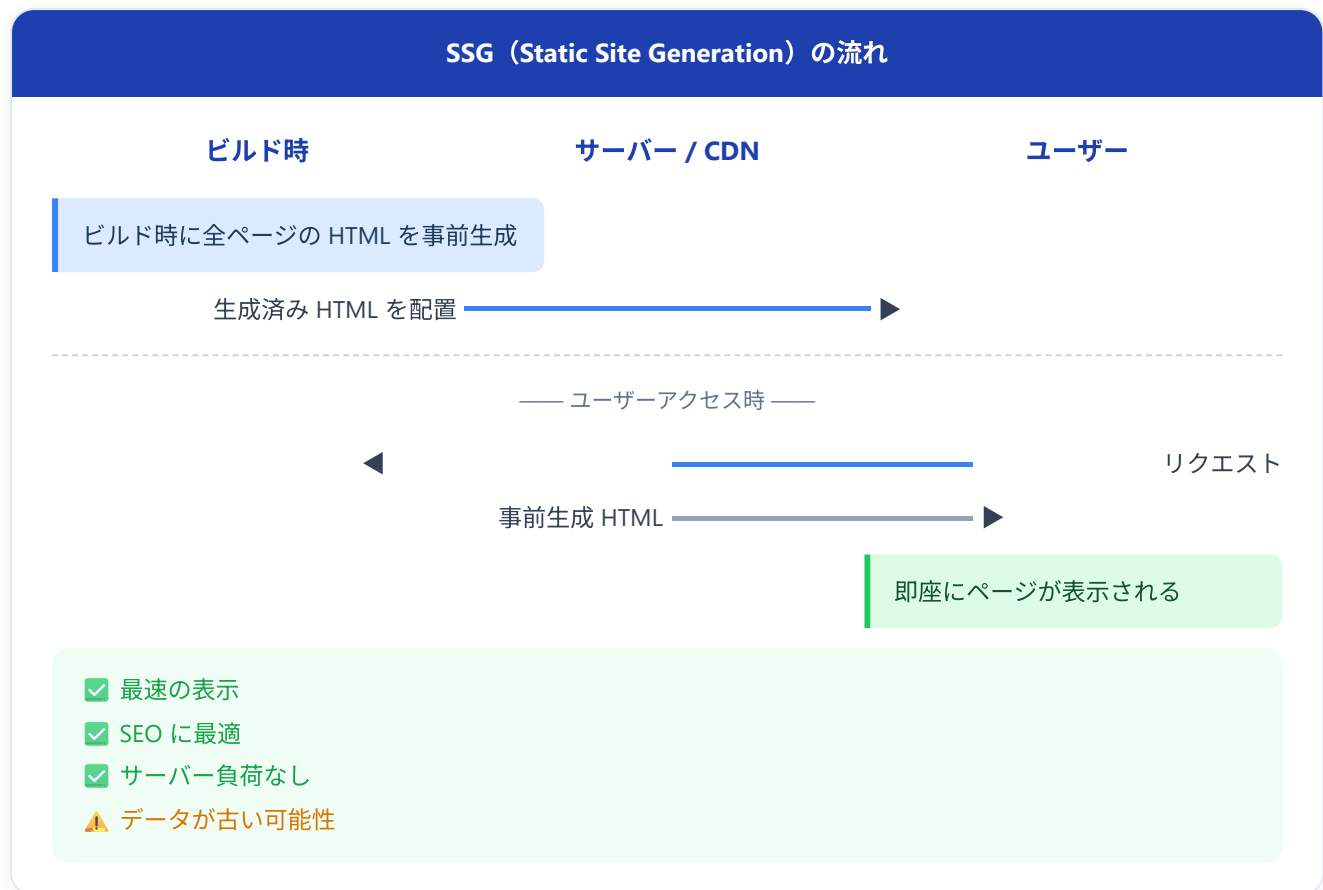
画面にはこう表示される: 最初に一瞬だけ白い画面やローディングスピナーが表示され、JavaScript の処理が完了してからコンテンツが表示されます。素の React (Create React App) はデフォルトでこの方式です。

SSR (Server-Side Rendering) - サーバサイドレンダリング



画面にはこう表示される: リクエストのたびにサーバーで HTML が生成されるため、最新のデータが反映された完成形のページがすぐに表示されます。ブラウザが JavaScript を読み込むと、ボタンクリックなどのインタラクションが可能になります (この過程を **Hydration** と呼びます)。

SSG（Static Site Generation） - 静的サイト生成



画面にはこう表示される: ビルド時に生成済みの HTML がそのまま返されるため、表示速度は最速です。ただし、ビルド後にデータが変わっても、再ビルドするまで古いデータが表示され続けます。ブログや企業サイトなど、更新頻度の低いページに最適です。

3つのレンダリング方式の比較

特性	CSR	SSR	SSG
初回表示速度	遅い	速い	最速
SEO	不利	有利	最も有利
データの鮮度	常に最新	常に最新	ビルド時点
サーバー負荷	低い	高い	最も低い
使用例	ダッシュボード	EC サイト商品ページ	ブログ記事

1.3 なぜ Next.js を使うのか（素の React との比較）

機能	素の React	Next.js
ルーティング	react-router 等を別途インストール	App Router が組み込み
SSR / SSG	自分で構築が必要（非常に複雑）	設定なしで利用可能
コード分割	手動で React.lazy 等を使う	自動で最適化
画像最適化	自分で実装	<code>next/image</code> が自動最適化
フォント最適化	自分で実装	<code>next/font</code> が自動最適化
API エンドポイント	Express 等の別サーバーが必要	Route Handlers / Server Actions
TypeScript	設定が必要	初期設定済み
ESLint	設定が必要	初期設定済み
環境変数	dotenv 等を別途設定	<code>.env.local</code> が組み込み
デプロイ	ホスティング先の選定・設定が必要	Vercel にワンクリックデプロイ

結論: 2024年以降、新規の React プロジェクトを始める場合は Next.js（App Router）を使うのがスタンダードです。React の公式ドキュメントでも Next.js が推奨フレームワークの1つとして紹介されています。

2. App Router

2.1 App Router vs Pages Router

Next.js には2つのルーティングシステムがあります。

特性	Pages Router (従来)	App Router (推奨)
ディレクトリ	<code>pages/</code>	<code>app/</code>
導入バージョン	Next.js 初期～	Next.js 13.4～ (安定版)
デフォルトコンポーネント	Client Component	Server Component
レイアウト	<code>_app.tsx</code> , <code>_document.tsx</code>	<code>layout.tsx</code> (ネスト可能)
データ取得	<code>getServerSideProps</code> , <code>getStaticProps</code>	async/await で直接
ローディング UI	自分で実装	<code>loading.tsx</code>
エラー UI	<code>_error.tsx</code> (グローバル)	<code>error.tsx</code> (ルートごと)
Server Actions	非対応	対応
Streaming	非対応	対応

本チュートリアルでは **App Router** のみを使用します。Pages Router は旧バージョンとの互換性のために残されていますが、新規プロジェクトでは App Router を選択してください。

2.2 ファイルベースルーティングの仕組み

Next.js の App Router では、`app/` ディレクトリ内のフォルダ構造がそのまま URL のパスに対応します。これが **ファイルベースルーティング** です。

ファイル構造 → URL パス

```

  app/
  ├── page.tsx → / (トップ)
  ├── about/
  │   └── page.tsx → /about
  ├── books/
  │   ├── page.tsx → /books
  │   ├── [id]/
  │   │   └── page.tsx → /books/123
  │   └── new/
  │       └── page.tsx → /books/new
  
```

重要なルール: - フォルダ がURLのセグメント (パスの一部) になる - `page.tsx` があるフォルダだけがアクセス可能なルートになる - `page.tsx` がないフォルダは URL として公開されない (コンポーネントの整理用に使える)

2.3 page.tsx, layout.tsx, loading.tsx, error.tsx の役割

App Router では、特別な名前を持つファイルにそれぞれ役割があります。

app/books/ ディレクトリ

layout.tsx (共通レイアウト)

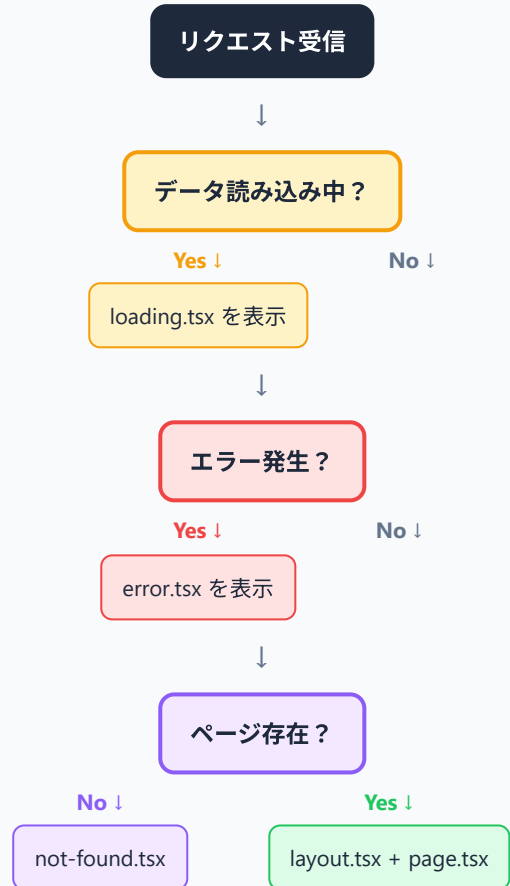
page.tsx (ページ本体)

loading.tsx (読み込み中の表示)

error.tsx (エラー時の表示)

not-found.tsx (404の表示)

レンダリングの流れ



page.tsx - ページ本体

そのルート（URL）にアクセスしたときに表示されるメインコンテンツです。

```
// app/page.tsx
// URL: /

export default function HomePage() {
  return (
    <div>
      <h1>書籍管理アプリへようこそ</h1>
      <p>あなたの読書記録を管理しましょう。</p>
    </div>
  );
}
```

画面にはこう表示される: ブラウザで `http://localhost:3000/` にアクセスすると、「書籍管理アプリへようこそ」という見出しと「あなたの読書記録を管理しましょう。」という段落が表示されます。

layout.tsx - 共通レイアウト

複数のページで共有される UI（ヘッダー、サイドバーなど）を定義します。ページ遷移してもレイアウトは再レンダリングされず、状態が保持されます。

```
// =====
// ファイルパス: app/layout.tsx
// 役割      : すべてのページを「ヘッダー+本文+フッター」の枠で包む
//            = Next.jsアプリの **一番外側の共通レイアウト**
// -----
// このファイルは Next.js が自動的に認識する「予約名」のひとつ。
// app/ フォルダ直下に layout.tsx が必ず1つ必要。
// =====

// `Metadata` 型を import する (Next.js 標準の型)。
// `import type` は「型だけ取り込む」記法。実行時のJSには残らないので軽量。
import type { Metadata } from "next";

// -----
// (1) メタデータの定義
// -----
// metadata という名前で export すると、Next.js が <head> 内の <title> や
// <meta name="description" ...> を自動生成してくれる。
// 自分で <head> を書く必要がない。
export const metadata: Metadata = {
  title: "書籍管理アプリ", // ブラウザタブの文字
  description: "あなたの読書記録を管理するアプリケーション", // SEO・SNS共有時の説明文
};

// -----
// (2) ルートレイアウトのコンポーネント本体
// -----
// `RootLayout` は予約名ではないが、慣習的にこの名前を付ける。
//
// 引数 (Props) :
//   - children: このレイアウトの中に差し込まれる「各ページの中身」
//   React.ReactNode 型は「JSX/文字列/数値/null/...などReactで扱える何でも」を表す。
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    // — 必須: <html> と <body> はルートレイアウトでだけ書く —
    // lang="ja" は「このページは日本語ですよ」という指定。
    // スクリーンリーダーや翻訳ツールがこれを参照する。
    <html lang="ja">
      <body>

        { /* — 共通ヘッダー (全ページに表示される) — */ }
        <header>
          <nav>
            <h1>  書籍管理アプリ </h1>
          </nav>

```

```

</header>

{/*
  — ページ本体が差し込まれる場所 —
  各 page.tsx の return 値が、ここに自動で入る。
  /books にアクセスしたら books/page.tsx の内容が、
  /books/123 にアクセスしたら books/[id]/page.tsx の内容がここに入る。
*/}

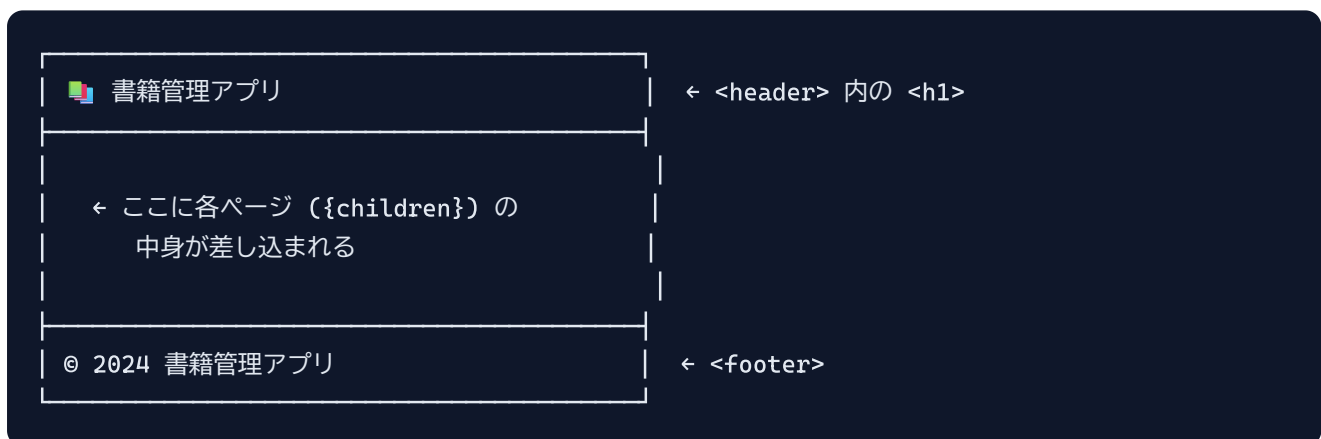
<main>{children}</main>

{/* — 共通フッター（全ページに表示される） — */}
<footer>
  <p>&copy; 2024 書籍管理アプリ</p>
</footer>

</body>
</html>
);
}

```

▼ ブラウザでの見た目（どのページでも上下にこの枠が出る）：



▼ 動作の仕組み:

URL	layout.tsx	中央 ({children})
/	表示される	app/page.tsx の中身
/books	表示される	app/books/page.tsx の中身
/books/123	表示される	app/books/[id]/page.tsx の中身

ページ遷移してもヘッダーとフッターは再描画されない（=スクロール位置や状態が保たれる）のが Next.js のレイアウト機能のメリットです。

loading.tsx - 読み込み中の表示

データの読み込み中に自動的に表示される UI です。React の `<Suspense>` を内部的に使用しています。

```
// app/books/loading.tsx

export default function BooksLoading() {
  return (
    <div className="loading-container">
      <div className="spinner" />
      <p>書籍データを読み込んでいます...</p>
    </div>
  );
}
```

画面にはこう表示される: `/books` にアクセスした直後、データベースからデータを取得している間、スピナーアニメーションと「書籍データを読み込んでいます...」というメッセージが表示されます。データの取得が完了すると、自動的に `page.tsx` の内容に切り替わります。

error.tsx - エラー時の表示

ページの描画中にエラーが発生した場合に表示される UI です。 `"use client"` が必須 です（エラーバウンダリは Client Component でなければならないため）。

```
// app/books/error.tsx
"use client";

export default function BooksError({
  error,
  reset,
}: {
  error: Error & { digest?: string };
  reset: () => void;
}) {
  return (
    <div className="error-container">
      <h2>エラーが発生しました</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>もう一度試す</button>
    </div>
  );
}
```

画面にはこう表示される: 書籍データの取得中にエラーが発生すると、「エラーが発生しました」という見出しとエラーメッセージ、そして「もう一度試す」ボタンが表示されます。ボタンを押すと、そのセグメントの再レンダリングが試みられます。

2.4 動的ルーティング ([id])

URL の一部をパラメータとして受け取りたい場合、フォルダ名を角括弧で囲みます。

```
app/  
  books/  
    [id]/  
      page.tsx    → /books/1, /books/2, /books/abc などにもマッチ
```

```
// app/books/[id]/page.tsx  
  
// params はオブジェクトとして渡される  
// Next.js 15 では params は Promise として渡されるため await が必要  
export default async function BookDetailPage({  
  params,  
}: {  
  params: Promise<{ id: string }>;  
}) {  
  const { id } = await params;  
  
  return (  
    <div>  
      <h1>書籍詳細</h1>  
      <p>書籍 ID: {id}</p>  
      { /* 実際にはここで id を使ってデータベースから書籍情報を取得する */ }  
    </div>  
  );  
}
```

画面にはこう表示される: - `/books/1` にアクセスすると「書籍 ID: 1」と表示されます。 - `/books/42` にアクセスすると「書籍 ID: 42」と表示されます。 - `/books/abc` にアクセスすると「書籍 ID: abc」と表示されます。

URL のその部分がそのまま `id` パラメータとして使えるわけです。

複数の動的セグメント

```
app/  
  users/  
    [userId]/  
      books/  
        [bookId]/  
          page.tsx    → /users/5/books/12 にマッチ
```

```
// =====
// ファイルパス: app/users/[userId]/books/[bookId]/page.tsx
// 役割      : URL 中の userId と bookId を取り出して表示するページ
// -----
// このファイルは Next.js が「動的セグメントを含むページ」と認識する。
// /users/5/books/12 にアクセスされると userId="5", bookId="12" になる。
// =====

// (1) 関数本体に async を付けると、内部で await が使える。
//     コンポーネント関数を async にできるのは Server Component だけの特権。
//
// (2) 引数の Props 型は { params: Promise<{ ... }> }
//     Next.js 15 以降では params が Promise でラップされて渡される。
//     なので await で「中身の値」を取り出してから使う。
export default async function UserBookPage({
  params,
}: {
  params: Promise<{ userId: string; bookId: string }>;
}) {
  // (3) Promise を await してオブジェクトを取り出し、
  //     さらに分割代入で userId と bookId に展開する。
  //     URL が /users/5/books/12 なら userId = "5", bookId = "12" となる。
  //     ※ 文字列で来ることに注意。数値が必要ななら Number(userId) で変換する。
  const { userId, bookId } = await params;

  // (4) 取り出した値を JSX 内に埋め込んで表示
  return (
    <div>
      <p>ユーザー ID: {userId}</p>
      <p>書籍 ID: {bookId}</p>
    </div>
  );
}

// ▼ /users/5/books/12 にアクセスしたときの画面表示:
//   ユーザー ID: 5
//   書籍 ID: 12
```

キャッチオールセグメント

```
app/
  docs/
    [...slug]/
      page.tsx → /docs/a, /docs/a/b, /docs/a/b/c などにマッチ
```

```
// =====
// ファイルパス: app/docs/[...slug]/page.tsx
// 役割      : スラッシュ区切りで何階層でもマッチする「キャッチオール」ページ
// -----
// [...slug] のように ... を付けると、それ以降のすべてのパスが配列として渡る。
// /docs/react          → slug = ["react"]
// /docs/react/hooks     → slug = ["react", "hooks"]
// /docs/react/hooks/useEffect → slug = ["react", "hooks", "useEffect"]
// =====

export default async function DocsPage({
  params,
}: {
  // (1) slug は文字列の配列として渡る
  params: Promise<{ slug: string[] }>;
}) {
  // (2) Promise から slug 配列を取り出す
  const { slug } = await params;
  // 例: /docs/react/hooks/useEffect → slug = ["react", "hooks", "useEffect"]

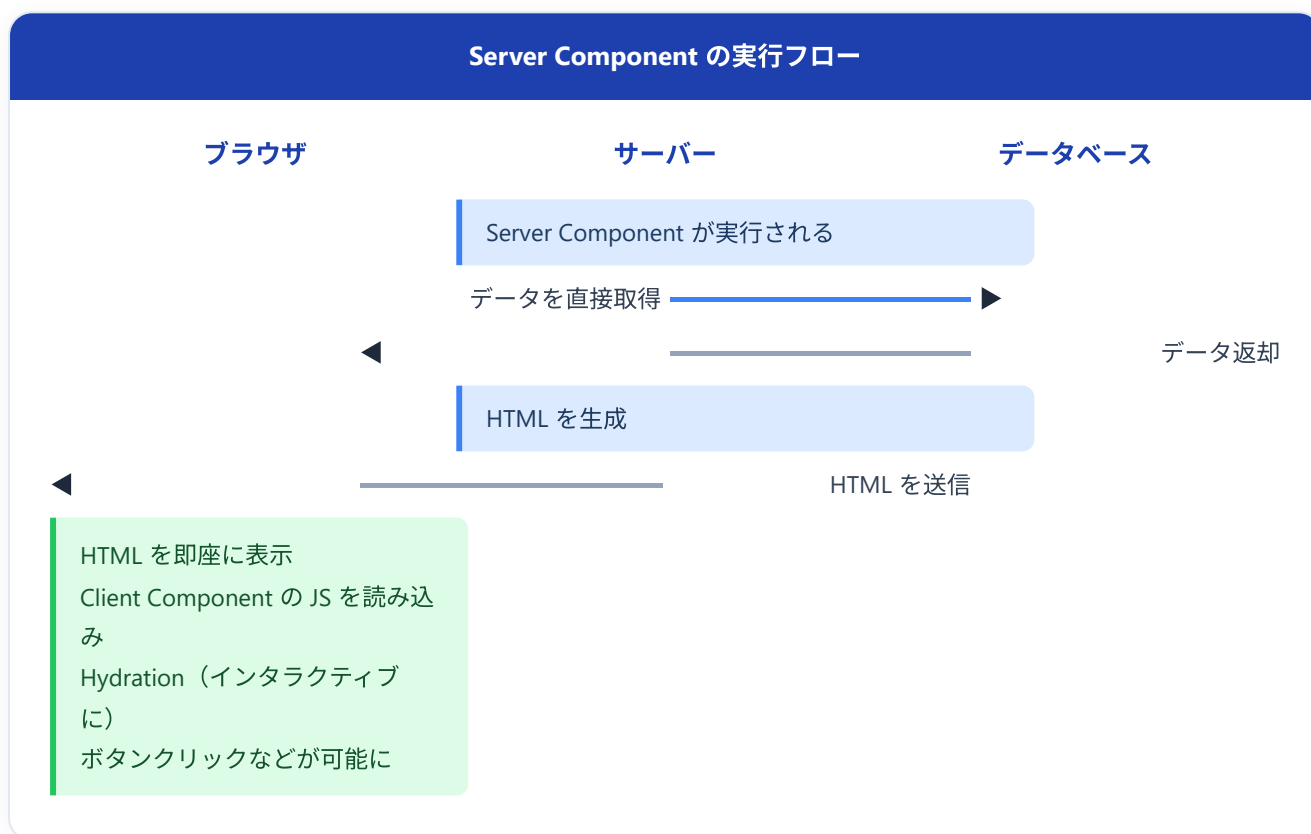
  // (3) Array.prototype.join(" / ") で配列を文字列にする
  // ["react", "hooks", "useEffect"].join(" / ") → "react / hooks / useEffect"
  return (
    <div>
      <p>パス: {slug.join(" / ")}</p>
    </div>
  );
}
// ▼ /docs/react/hooks/useEffect にアクセスしたときの画面表示:
//   パス: react / hooks / useEffect
```

3. Server Components vs Client Components

3.1 概念の説明

Next.js App Router の最大の特徴は、デフォルトですべてのコンポーネントが **Server Component** であるということです。

Server Components (デフォルト)	Client Components ('use client')
✓ サーバー上で実行される ✓ データベースに直接アクセス可能 ✓ JS バンドルに含まれない ✗ useState, useEffect 使用不可 ✗ onClick 等のイベント処理不可 ✗ ブラウザ API 使用不可	🌐 ブラウザ上で実行される 🌐 API 経由でデータ取得 🌐 JS バンドルに含まれる ✓ useState, useEffect 使用可能 ✓ onClick 等のイベント処理可能 ✓ ブラウザ API 使用可能



3.2 "use client" ディレクティブ

Client Component にするには、ファイルの **先頭** に `"use client"` と記述します。

```
// =====
// Server Component の例 ("use client" を書かない=デフォルト)
// =====
// このファイルはサーバーでだけ実行される。
//   ✓ 関数を async にして await でDBやAPIにアクセスできる
//   ✓ 機密情報 (APIキーなど) を扱える (ブラウザに送られない)
//   ✗ useState / useEffect / onClick などのインタラクティブ機能は使えない
//   ✗ window / document などブラウザ専用のAPIは使えない

export default function BookList() {
  // (1) 通常はここで await fetch(...) や DBクライアントを呼んでデータを取得する
  //   例: const books = await db.query("SELECT * FROM books");
  return <div>書籍一覧 (サーバーで描画) </div>;
}
```

```
// =====
// Client Component の例 ("use client" をファイル先頭に書く)
// =====
// このディレクティブが書かれた瞬間から、このファイルとそこから import される
// すべてのコンポーネントはブラウザで動く扱いになる。
"use client";

// (1) Client Component ではブラウザ側で動く React のフックが使える
import { useState } from "react";

export default function SearchBar() {
  // (2) useState で「検索クエリ文字列」を状態として管理する
  //   [現在の値, 値を変える関数] = useState(初期値)
  //   初期値が "" なので最初の query は空文字。
  const [query, setQuery] = useState("");

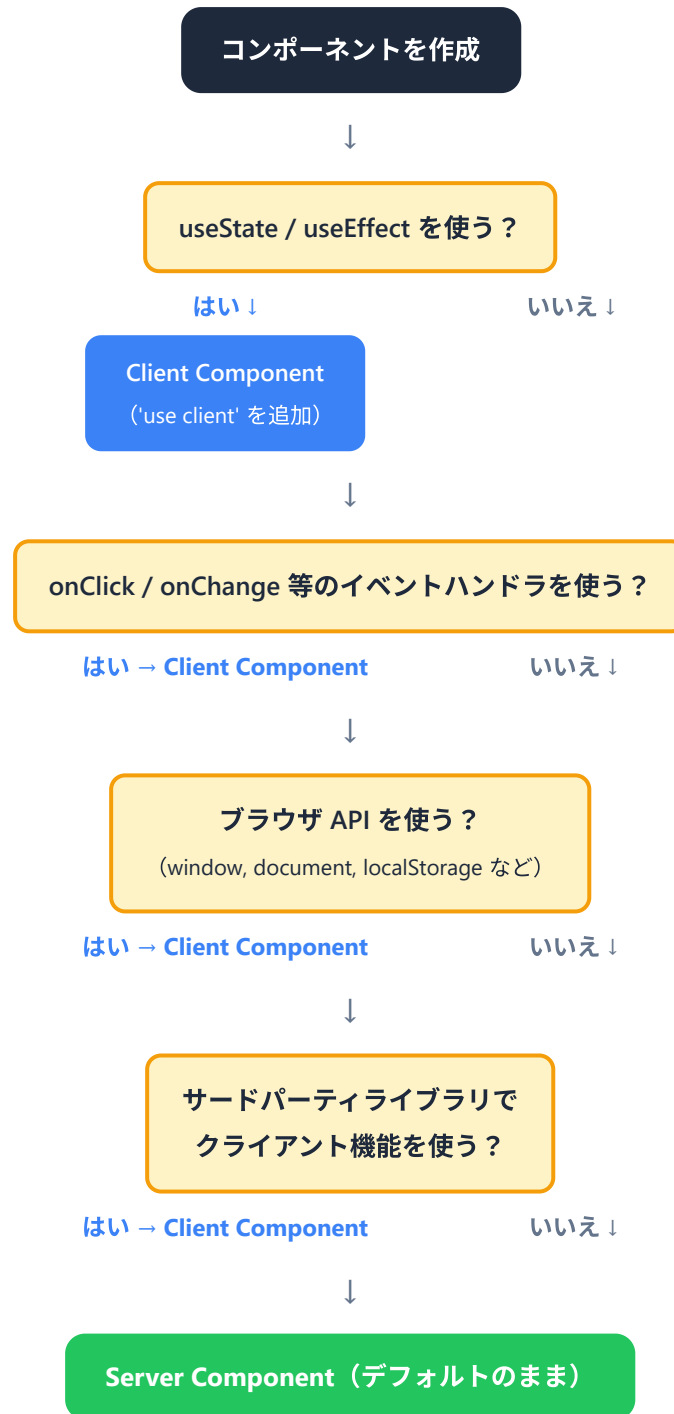
  // (3) JSX で <input> を描画
  //   ・value={query}      : 入力欄に現在の状態を反映する (制御コンポーネント)
  //   ・onChange={e=>...}: ユーザーが文字を打つたびに呼ばれる関数
  //   ・e.target.value     : 入力欄の現在の値 (input要素のテキスト)
  //   ・setQuery(...)     : 状態を更新 → React が再描画 → value も更新される
  return (
    <input
      type="text"
      value={query}
      onChange={e => setQuery(e.target.value)}
      placeholder="書籍を検索..."
    />
  );
}
```

▼ 動作:

- 検索バー（テキスト入力欄）が表示される
- キーボードで「Re」と打つと query が "" → "R" → "Re" に更新され、画面の入力欄もそれに追従する
- これは `useState` がブラウザのメモリに値を保持し、変更ごとに React が再描画するため
- サーバーはこの入力プロセスにまったく関与しない

重要: `"use client"` はそのファイルとそこから import されるすべてのモジュールを Client Component の境界として宣言します。つまり、Client Component の子コンポーネントも自動的に Client Component になります。

3.3 いつどちらを使うか（判断フローチャート）



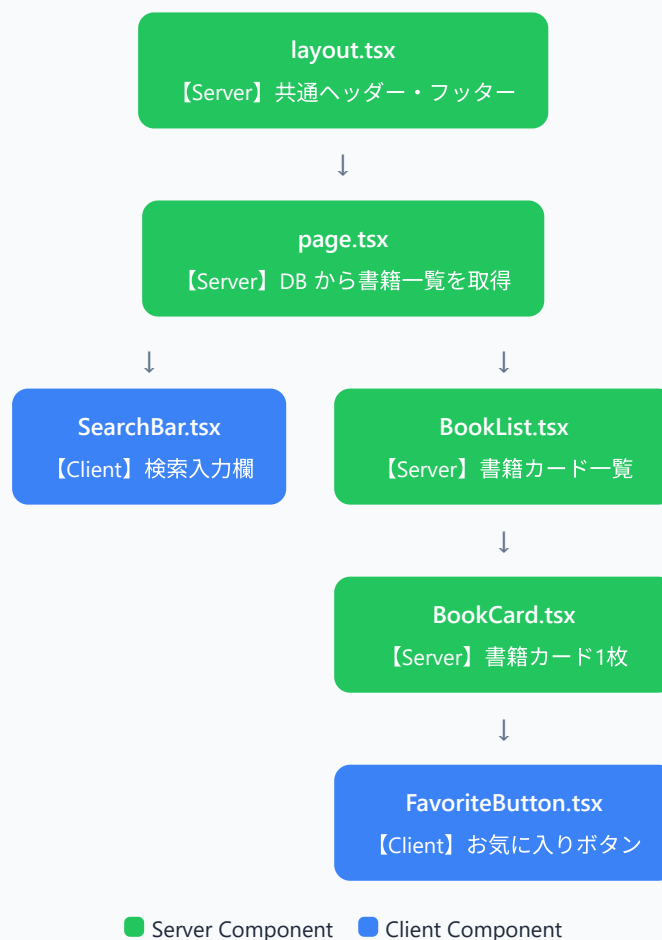
原則: できるだけ Server Component を使い、インタラクティブ性が必要な部分だけを Client Component にします。

用途	Server Component	Client Component
データの取得・表示	推奨	
データベースへの直接アクセス	推奨	不可
機密情報（APIキーなど）へのアクセス	推奨	不可
静的な UI の描画	推奨	
フォーム入力		必須
クリックイベント		必須
useState / useEffect		必須
ブラウザ API（localStorage 等）	不可	必須

3.4 書籍管理アプリでの使い分け例

書籍管理アプリの具体的なページで、どのようにコンポーネントを分割するか見てみましょう。

書籍一覧ページ /books — コンポーネント構成



```

// app/books/page.tsx (Server Component)
// データベースから直接書籍一覧を取得する

import { SearchBar } from "@components/SearchBar";
import { BookList } from "@components/BookList";

export default async function BooksPage() {
  // Server Component なので、サーバー上で直接データ取得ができる
  // この処理はブラウザには送信されない
  const response = await fetch("https://api.example.com/books", {
    cache: "no-store", // 常に最新データを取得 (SSR)
  });
  const books = await response.json();

  return (
    <div>
      <h1>書籍一覧</h1>
      { /* Client Component: ユーザーの入力を受け付ける */ }
      <SearchBar />
      { /* Server Component: 取得したデータを表示する */ }
      <BookList books={books} />
    </div>
  );
}

```

```

// components/SearchBar.tsx (Client Component)
"use client";

import { useState } from "react";
import { useRouter } from "next/navigation";

export function SearchBar() {
  const [query, setQuery] = useState("");
  const router = useRouter();

  const handleSearch = (e: React.FormEvent) => {
    e.preventDefault();
    // 検索クエリ付きで遷移
    router.push(`/books?q=${encodeURIComponent(query)}`);
  };

  return (
    <form onSubmit={handleSearch}>
      <input
        type="text"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="タイトルで検索..."
      />
      <button type="submit">検索</button>
    </form>
  );
}

```

```
// components/BookCard.tsx (Server Component)

import { FavoriteButton } from "../FavoriteButton";

type Book = {
  id: string;
  title: string;
  author: string;
};

export function BookCard({ book }: { book: Book }) {
  return (
    <div className="book-card">
      <h3>{book.title}</h3>
      <p>{book.author}</p>
      { /* インタラクティブな部分だけ Client Component */ }
      <FavoriteButton bookId={book.id} />
    </div>
  );
}
```

```
// components/FavoriteButton.tsx (Client Component)
"use client";

import { useState } from "react";

export function FavoriteButton({ bookId }: { bookId: string }) {
  const [isFavorite, setIsFavorite] = useState(false);

  const handleClick = async () => {
    setIsFavorite(!isFavorite);
    // API を呼び出してお気に入り状態を保存
    await fetch(`/api/books/${bookId}/favorite`, {
      method: "POST",
      body: JSON.stringify({ favorite: !isFavorite }),
    });
  };

  return (
    <button onClick={handleClick}>
      {isFavorite ? "★ お気に入り済み" : "☆ お気に入り"}
    </button>
  );
}
```

画面にはこう表示される: ページ上部に検索バー、その下に書籍カードが並んで表示されます。各カードには書籍タイトル、著者名、お気に入りボタンがあります。検索バーに文字を入力するとリアルタイムに反映され、お気に入りボタンをクリックすると「☆ お気に入り」が「★ お気に入り済み」に切り替わります。

4. レイアウトとテンプレート

4.1 ルートレイアウト

`app/layout.tsx` は ルートレイアウト と呼ばれ、アプリケーション全体に適用されます。 `<html>` タグと `<body>` タグを含む 唯一の レイアウトです。

```
// app/layout.tsx

import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";

// Google Fonts の Inter フォントを最適化して読み込む
const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
  title: {
    template: "%s | 書籍管理アプリ",
    default: "書籍管理アプリ",
  },
  description: "あなたの読書記録を管理するアプリケーション",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="ja">
      <body className={inter.className}>
        {/* ここにヘッダーを配置 */}
        <header className="site-header">
          <nav>
            <a href="/">書籍管理アプリ</a>
          </nav>
        </header>

        {/* children に各ページの内容が入る */}
        <main className="site-main">{children}</main>

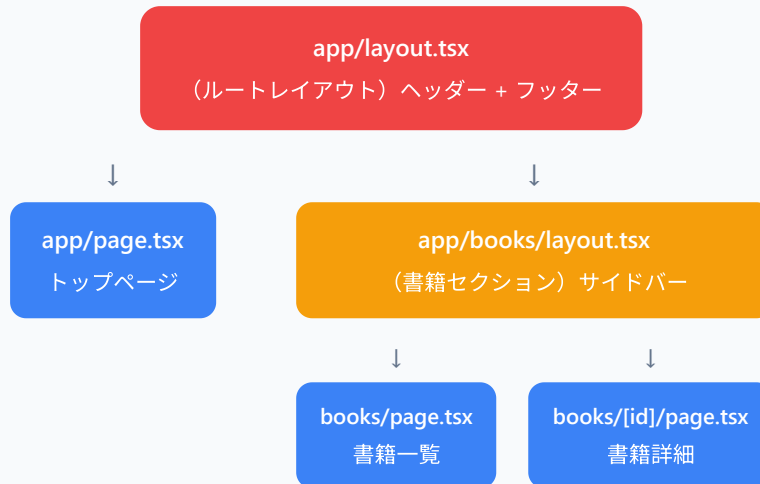
        {/* ここにフッターを配置 */}
        <footer className="site-footer">
          <p>&copy; 2024 書籍管理アプリ</p>
        </footer>
      </body>
    </html>
  );
}
```

ポイント: - ルートレイアウトは **必須** です。削除するとエラーになります。 - `<html>` と `<body>` タグはルートレイアウトにのみ記述します。 - `metadata` オブジェクトで SEO 用のタイトル・説明を設定できます。 - `template: "%s | 書籍管理アプリ"` により、子ページのタイトルが「ページ名 | 書籍管理アプリ」の形式になります。

4.2 ネストされたレイアウト

各ルートセグメント（フォルダ）にも `layout.tsx` を置くことができ、そのセグメント以下のページに適用されます。

レイアウトのネスト構造



```
// app/books/layout.tsx
// /books 以下のすべてのページに適用されるレイアウト

import type { Metadata } from "next";

export const metadata: Metadata = {
  title: "書籍管理",
};

export default function BooksLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <div className="books-layout">
      {/* サイドバー */}
      <aside className="sidebar">
        <nav>
          <ul>
            <li><a href="/books">書籍一覧</a></li>
            <li><a href="/books/new">書籍を追加</a></li>
            <li><a href="/books/favorites">お気に入り</a></li>
          </ul>
        </nav>
      </aside>

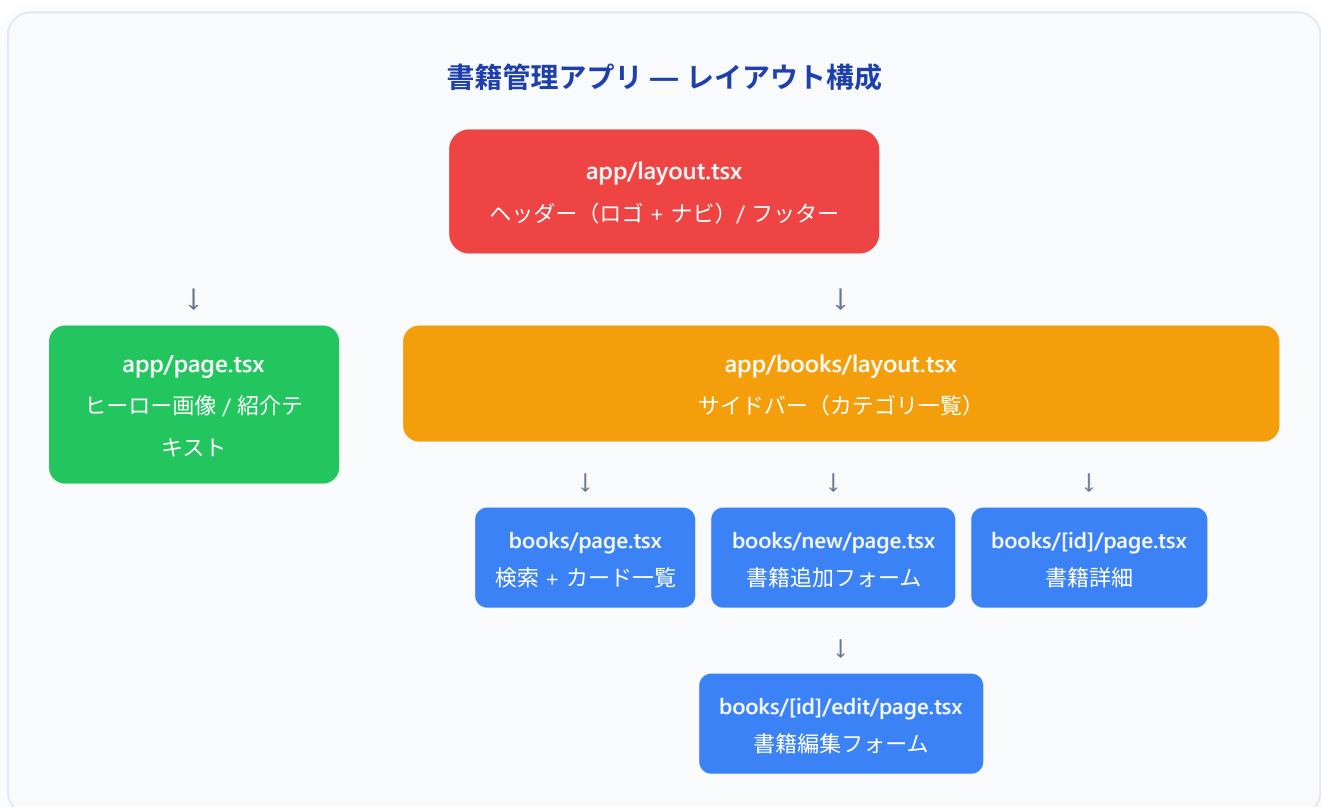
      {/* メインコンテンツ */}
      <section className="content">
        {children}
      </section>
    </div>
  );
}
```

画面にはこう表示される:



`/books` や `/books/123` にアクセスすると、ルートレイアウトのヘッダー・フッターに加えて、書籍セクション専用のサイドバーが表示されます。トップページ（`/`）にはサイドバーは表示されません。

4.3 書籍管理アプリのレイアウト構成



5. ナビゲーション

5.1 Link コンポーネント

Next.js では、ページ間の遷移に HTML の `<a>` タグではなく、`next/link` の `<Link>` コンポーネントを使います。

```
// components/Navigation.tsx
import Link from "next/link";

export function Navigation() {
  return (
    <nav>
      <ul>
        <li>
          <Link href="/">ホーム</Link>
        </li>
        <li>
          <Link href="/books">書籍一覧</Link>
        </li>
        <li>
          <Link href="/books/new">書籍を追加</Link>
        </li>
      </ul>
    </nav>
  );
}
```

`<Link>` と `<a>` の違い:

特性	<code><a></code> タグ	<code><Link></code> コンポーネント
ページ遷移	ページ全体をリロード	クライアント側で遷移（高速）
プリフェッチ	なし	ビューポート内のリンクを自動プリフェッチ
状態の保持	リセットされる	レイアウトの状態が保持される
JavaScript	不要	必要

```
// 動的ルートへのリンク
import Link from "next/link";

type Book = {
  id: string;
  title: string;
};

export function BookLink({ book }: { book: Book }) {
  return (
    <Link href={`/${books}/${book.id}`}>
      {book.title} の詳細を見る
    </Link>
  );
}
```

画面にはこう表示される:「〇〇の詳細を見る」というリンクテキストが表示されます。クリックすると、ページ全体がリロードされることなく、スムーズに書籍詳細ページに遷移します。ヘッダーやサイドバーのレイアウトはそのまま残り、メインコンテンツ部分だけが切り替わります。

プリフェッチについて: `<Link>` コンポーネントはデフォルトで、ユーザーの画面（ビューポート）に表示されているリンク先のページを裏で先読みします。これにより、リンクをクリックした瞬間にページが表示されるような高速な体験が実現します。

5.2 useRouter フック

プログラムからページ遷移を行いたい場合（ボタンクリック後やフォーム送信後など）は、`useRouter` フックを使います。Client Component でのみ使用可能です。

```
// components/BookForm.tsx
"use client";

import { useRouter } from "next/navigation"; // ⚠️ next/router ではない!

export function BookForm() {
  const router = useRouter();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();

    // 書籍を保存する処理 (省略)

    // 保存後、書籍一覧ページに遷移
    router.push("/books");
  };

  return (
    <form onSubmit={handleSubmit}>
      {/* フォームの内容 (省略) */}
      <button type="submit">保存</button>
    </form>
  );
}
```

useRouter の主要メソッド:


```

"use client";

import { useRouter } from "next/navigation";

export function NavigationExample() {
  const router = useRouter();

  return (
    <div>
      {/* 指定したパスに遷移（履歴に追加） */}
      <button onClick={() => router.push("/books")}>
        書籍一覧へ
      </button>

      {/* 指定したパスに遷移（履歴を置換） */}
      <button onClick={() => router.replace("/books")}>
        書籍一覧へ（履歴を置換）
      </button>

      {/* ブラウザの「戻る」と同じ */}
      <button onClick={() => router.back()}>
        戻る
      </button>

      {/* ページのデータを再取得（再レンダリング） */}
      <button onClick={() => router.refresh()}>
        更新
      </button>
    </div>
  );
}

```

メソッド	説明	使用例
<code>router.push(url)</code>	指定した URL に遷移	フォーム送信後のリダイレクト
<code>router.replace(url)</code>	現在の履歴エントリを置換して遷移	ログイン後のリダイレクト
<code>router.back()</code>	ブラウザの戻るボタンと同じ	詳細ページから一覧に戻る
<code>router.refresh()</code>	現在のページを再取得	データ更新後の再読み込み
<code>router.prefetch(url)</code>	指定した URL を先読み	近く遷移しそうなページの準備

5.3 リダイレクト

Server Component でのリダイレクト

```
// app/books/[id]/page.tsx
import { redirect, notFound } from "next/navigation";

export default async function BookDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const book = await fetchBook(id);

  // 書籍が見つからない場合は 404 ページを表示
  if (!book) {
    notFound();
  }

  // 非公開の書籍は一覧ページにリダイレクト
  if (book.isPrivate) {
    redirect("/books");
  }

  return (
    <div>
      <h1>{book.title}</h1>
    </div>
  );
}

async function fetchBook(id: string) {
  // データベースから書籍を取得する処理 (仮)
  return { title: "サンプル書籍", isPrivate: false };
}
```

middleware.ts でのリダイレクト

`middleware.ts` をプロジェクトルートに置くことで、リクエストの段階でリダイレクトを行えます。認証チェックなどに便利です。

```
// middleware.ts (プロジェクトルート)

import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
  // 例: 未ログインユーザーを /books ページからリダイレクト
  const isLoggedIn = request.cookies.get("session");

  if (!isLoggedIn && request.nextUrl.pathname.startsWith("/books")) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  return NextResponse.next();
}

// ミドルウェアを適用するパスを指定
export const config = {
  matcher: ["/books/:path*"],
};
```

6. データフェッチ

6.1 Server Component でのデータ取得

Server Component ではコンポーネント関数を `async` にして、直接 `await` でデータを取得できます。これが Next.js App Router の最も強力な機能の1つです。

```
// =====
// ファイルパス: app/books/page.tsx
// 役割      : 書籍一覧ページ。サーバーで全書籍を取ってきてHTMLにする。
// 種類      : Server Component ("use client" を書いていないので自動でこちら)
// -----
// Server Component の最大の魅力は「コンポーネント関数自体を async にできて、
// 直接 await でデータを取ってこられる」こと。
// そのデータはサーバーで埋め込まれた状態でブラウザに届くので、
// 初回表示が速く、SEOにも有利。
// =====

// (1) 書籍データ1件分の「形」を定義
// Book 型を1度ここで決めておくと、map のコールバック内などで
// book.〇〇 と書いたとき VS Code が補完してくれる。
type Book = {
  id: string;           // 一意なID
  title: string;        // 書籍タイトル
  author: string;       // 著者名
  publishedYear: number; // 出版年
};

// (2) APIから書籍配列を取ってくる関数
// async を付けると、関数の中で await が使えるようになる。
// 戻り値は「Book型の配列が将来届く」を意味する Promise<Book[]>。
async function getBooks(): Promise<Book[]> {

  // fetch は Web標準のHTTP通信API。
  // 第2引数の cache オプションで Next.js のキャッシュ動作を指定する。
  const response = await fetch("https://api.example.com/books", {
    cache: "no-store", // ← 毎リクエストで最新データを取得する (=SSR動作)
    // cache: "force-cache", // ← ビルド時に1度だけ取得 (=SSG動作)
    // next: { revalidate: 60 }, // ← 60秒ごとに再取得 (=ISR動作)
  });

  // (2-1) 通信は成功したけどステータスが 4xx/5xx の場合は失敗扱いにする。
  // response.ok は status が 200~299 のとき true、それ以外で false。
  if (!response.ok) {
    throw new Error("書籍データの取得に失敗しました");
  }

  // (2-2) JSON 文字列を JS オブジェクトに変換して返す
  return response.json();
}

// (3) ページコンポーネント本体
// 関数自体に async を付けられるのが Server Component の特権。
```

```
// Client Component ("use client"あり) では関数本体を async にできない。
export default async function BooksPage() {

  // この行はサーバーで実行される。
  // 取得処理が終わるまでブラウザにはHTMLが送られない（その間 loading.tsx が表示される）。
  const books = await getBooks();

  // (4) 取れたデータを使ってJSXを組み立てる
  // books.length は配列の要素数（書籍が3件なら3）。
  // books.map((book) => ...) で1件ずつJSXに変換する。
  // key={book.id} は React がリストの差分検出に使う必須の属性。
  return (
    <div>
      <h1>書籍一覧 ({books.length}冊) </h1>
      <ul>
        {books.map((book) => (
          <li key={book.id}>
            <h3>{book.title}</h3>
            <p>著者: {book.author} ({book.publishedYear}年) </p>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

▼ APIが3件のデータを返した場合のブラウザ表示:

書籍一覧（3冊）

- ・ リーダブルコード
著者: Dustin Boswell (2012年)
- ・ プロを目指す人のためのTypeScript入門
著者: 鈴木 僚太 (2022年)
- ・ 達人プログラマー
著者: David Thomas (2016年)

▼ 通信失敗時:

`throw new Error(...)` が走ると、Next.js は同じセグメント内の `error.tsx` を自動で表示します。 `error.tsx` を作っていない場合は親のエラーバウンダリにバブリングします。

▼ ブラウザに届くHTMLの様子（要点だけ抜粋）:

```

<div>
  <h1>書籍一覧（3冊）</h1>
  <ul>
    <li><h3>リーダブルコード</h3><p>著者：Dustin Boswell（2012年）</p></li>
    <li><h3>プロを目指す人のためのTypeScript入門</h3><p>著者：鈴木 僚太（2022年）</p></li>
    <li><h3>達人プログラマー</h3><p>著者：David Thomas（2016年）</p></li>
  </ul>
</div>

```

▼ 普通のReactとの最大の違い: クライアントサイドの `useState/useEffect` でデータを取りに行くのと違って、最初からデータが入ったHTMLが返ってくるので、表示がカクカクしない・SEOに強い・JSバンドルも小さく済む。

fetch のキャッシュ戦略

```

// SSR - リクエストのたびに最新データを取得
fetch(url, { cache: "no-store" });

// SSG - ビルド時にデータを取得し、キャッシュ（デフォルト）
fetch(url, { cache: "force-cache" });

// ISR - 60秒ごとにキャッシュを再検証
fetch(url, { next: { revalidate: 60 } });

```

戦略	説明	使用例
<code>cache: "no-store"</code>	毎回取得（SSR）	ユーザー固有のデータ
<code>cache: "force-cache"</code>	ビルド時に取得（SSG）	変更の少ないデータ
<code>next: { revalidate: N }</code>	N秒ごとに再検証（ISR）	ニュース記事など

6.2 Client Component でのデータ取得

Client Component では、従来の React と同様に `useEffect` と `useState` を使うか、サードパーティのデータ取得ライブラリ（SWR や TanStack Query）を使います。

```

// components/BookSearch.tsx
"use client";

import { useState, useEffect } from "react";

type Book = {
  id: string;
  title: string;
  author: string;
};

export function BookSearch() {
  const [query, setQuery] = useState("");
  const [books, setBooks] = useState<Book[]>([]);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // 検索クエリが空なら何もしない
    if (!query.trim()) {
      setBooks([]);
      return;
    }

    // デバウンス: 入力が止まって300ms後に検索を実行
    const timer = setTimeout(async () => {
      setIsLoading(true);
      setError(null);

      try {
        const response = await fetch(
          `/api/books/search?q=${encodeURIComponent(query)}`
        );
        if (!response.ok) {
          throw new Error("検索に失敗しました");
        }
        const data = await response.json();
        setBooks(data);
      } catch (err) {
        setError(err instanceof Error ? err.message : "エラーが発生しました");
      } finally {
        setIsLoading(false);
      }
    }, 300);

    // クリーンアップ: 次の入力が来たらタイマーをキャンセル
    return () => clearTimeout(timer);
  }, [query]);

```

```

return (
  <div>
    <input
      type="text"
      value={query}
      onChange={e => setQuery(e.target.value)}
      placeholder="書籍を検索..."
    />

    {isLoading && <p>検索中...</p>}
    {error && <p className="error">{error}</p>}}

    <ul>
      {books.map((book) => (
        <li key={book.id}>
          {book.title} - {book.author}
        </li>
      ))}
    </ul>
  </div>
);
}

```

画面にはこう表示される: 検索入力欄が表示されます。ユーザーが「村上」と入力すると、入力が止まって300ミリ秒後に「検索中...」というメッセージが一瞬表示され、その後「村上春樹」の著書一覧がリスト形式で表示されます。

Server Component と Client Component のデータ取得の比較:

特性	Server Component	Client Component
コード量	少ない (async/await のみ)	多い (useState + useEffect)
ローディング状態	loading.tsx が自動管理	自分で管理が必要
エラー処理	error.tsx が自動管理	自分で管理が必要
データの安全性	APIキー等が露出しない	APIキーは使えない
SEO	HTML にデータが含まれる	初回は空
リアルタイム更新	ページ再読み込みが必要	可能

6.3 loading.tsx による読み込み状態

`loading.tsx` を配置するだけで、Server Component のデータ取得中に自動的にローディング UI が表示されます。

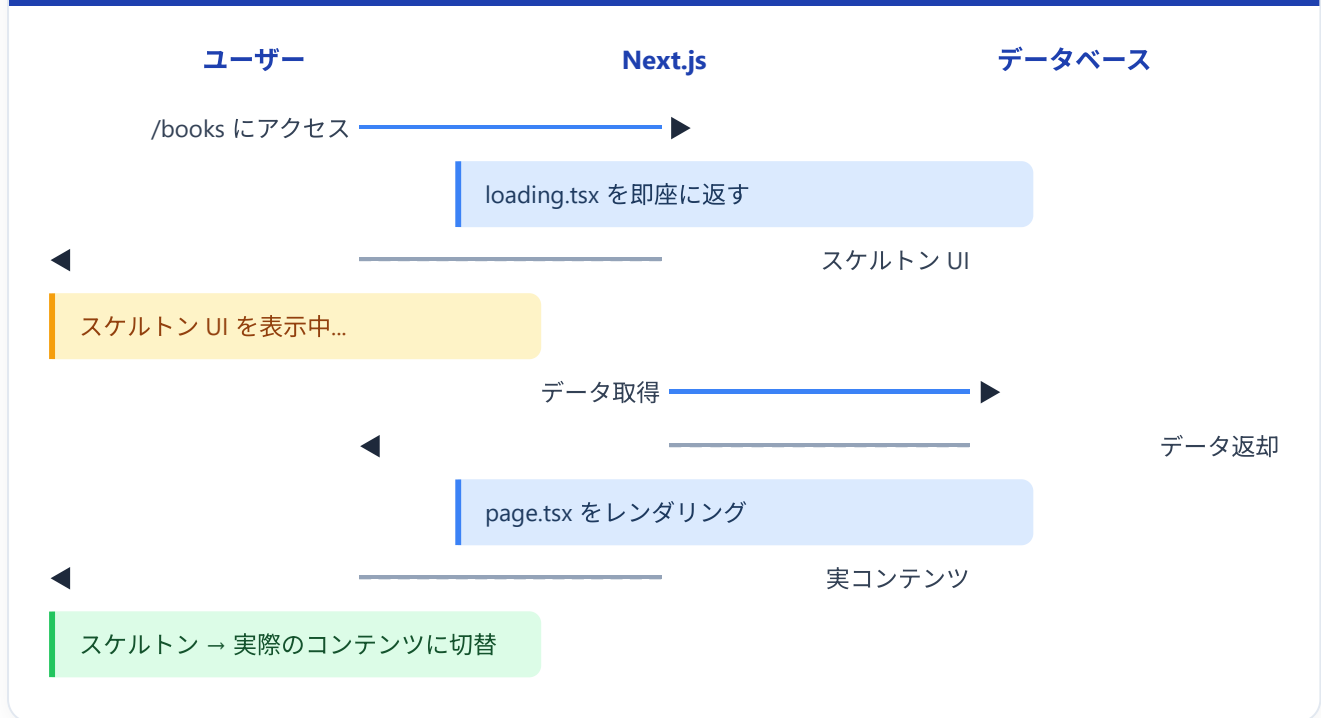

```
// app/books/loading.tsx

export default function BooksLoading() {
  return (
    <div className="loading-container">
      {/* スケルトンUI: 実際のコンテンツと同じ形状の灰色の枠 */}
      <h1 className="skeleton" style={{ width: "200px", height: "32px" }} />

      <div className="book-grid">
        {/* 6個のスケルトンカード */}
        {Array.from({ length: 6 }).map((_, i) => (
          <div key={i} className="skeleton-card">
            <div
              className="skeleton"
              style={{ width: "100%", height: "200px" }}
            />
            <div
              className="skeleton"
              style={{ width: "80%", height: "20px", marginTop: "8px" }}
            />
            <div
              className="skeleton"
              style={{ width: "60%", height: "16px", marginTop: "4px" }}
            />
          </div>
        ))}
      </div>
    </div>
  );
}
```

画面にはこう表示される: データの読み込み中、見出しの位置に灰色のバーが1つ、その下に灰色のカード状のブロックが6個並びます。これらは実際のコンテンツと同じ大きさ・位置で表示されるため、読み込みが完了するとスムーズに実際のコンテンツに切り替わり、画面がガタつきません（この手法を **スケルトンUI** と呼びます）。

loading.tsx による Streaming の流れ



7. Server Actions

7.1 Server Actions とは

Server Actions は、Client Component から直接サーバー上の関数を呼び出せる 仕組みです。API ルートを別途作成する必要がなく、フォーム処理やデータの変更（作成・更新・削除）をシンプルに書けます。

従来の方法

Client Component
(フォーム)

`fetch('/api/books')`
→

API Route
(route.ts)

DB操作
→

データベース

Server Actions (新しい方法)

Client Component
(フォーム)

関数を直接呼び出し
→

Server Action
(サーバー上で実行)

DB操作
→

データベース

Server Actions は `"use server"` ディレクティブで宣言します。

7.2 フォーム処理での利用

基本的な Server Action

```
// app/books/new/actions.ts
"use server";

// この関数はサーバー上でのみ実行される
// クライアントには関数の中身は送信されない
export async function createBook(formData: FormData) {
  const title = formData.get("title") as string;
  const author = formData.get("author") as string;
  const publishedYear = Number(formData.get("publishedYear"));

  // バリデーション
  if (!title || !author) {
    return {
      error: "タイトルと著者は必須です",
    };
  }

  // データベースに保存 (例)
  // const book = await db.book.create({
  //   data: { title, author, publishedYear },
  // });

  console.log("書籍を追加:", { title, author, publishedYear });

  // 成功したら書籍一覧にリダイレクト
  // redirect("/books");
  return { success: true };
}
```

Server Component のフォーム (JavaScript 不要)

```
// app/books/new/page.tsx (Server Component)

import { createBook } from "../actions";

export default function NewBookPage() {
  return (
    <div>
      <h1>書籍を追加</h1>

      {/* action 属性に Server Action を渡す */}
      {/* JavaScript が無効でも動作する (Progressive Enhancement) */}
      <form action={createBook}>
        <div>
          <label htmlFor="title">タイトル</label>
          <input
            type="text"
            id="title"
            name="title"
            required
          />
        </div>

        <div>
          <label htmlFor="author">著者</label>
          <input
            type="text"
            id="author"
            name="author"
            required
          />
        </div>

        <div>
          <label htmlFor="publishedYear">出版年</label>
          <input
            type="number"
            id="publishedYear"
            name="publishedYear"
          />
        </div>

        <button type="submit">追加する</button>
      </form>
    </div>
  );
}
```

画面にはこう表示される: 「書籍を追加」という見出しの下に、タイトル、著者、出版年の入力フィールドと「追加する」ボタンが表示されます。各フィールドに入力して「追加する」ボタンを押すと、フォームのデータがサーバーに送信され、`createBook` 関数がサーバー上で実行されます。

Client Component のフォーム（ローディング状態の管理）

```
// components/BookFormClient.tsx
"use client";

import { useActionState } from "react";
import { createBook } from "@app/books/new/actions";

export function BookFormClient() {
  const [state, formAction, isPending] = useActionState(createBook, null);

  return (
    <form action={formAction}>
      <div>
        <label htmlFor="title">タイトル</label>
        <input type="text" id="title" name="title" required />
      </div>

      <div>
        <label htmlFor="author">著者</label>
        <input type="text" id="author" name="author" required />
      </div>

      <div>
        <label htmlFor="publishedYear">出版年</label>
        <input type="number" id="publishedYear" name="publishedYear" />
      </div>

      {/* エラーメッセージの表示 */}
      {state?.error && (
        <p className="error-message">{state.error}</p>
      )}

      {/* 送信中はボタンを無効化 */}
      <button type="submit" disabled={isPending}>
        {isPending ? "追加中..." : "追加する"}
      </button>
    </form>
  );
}
```

画面にはこう表示される: フォームは前の例と同じ見た目ですが、「追加する」ボタンを押すと、ボタンのテキストが「追加中...」に変わり、ボタンがグレイアウトして再クリックできなくなります。バリデーションエラーがあれば、ボタンの上にエラーメッセージが赤字で表示されます。

7.3 書籍の追加/編集での利用予告

次章以降で、Server Actions を使って以下の機能を実装します:

書籍管理アプリの Server Actions

createBook
書籍の新規追加

updateBook
書籍情報の編集

deleteBook
書籍の削除

toggleFavorite
お気に入り切替

searchBooks
書籍の検索

createBook

- フォームデータの取得
- バリデーション
- Supabase に INSERT
- /books にリダイレクト

updateBook

- フォームデータの取得
- バリデーション
- Supabase を UPDATE
- /books/[id] にリダイレクト

deleteBook

- 確認チェック
- Supabase から DELETE
- /books にリダイレクト

予告: 第6章「Supabase との連携」で、実際のデータベースと接続してこれらの Server Actions を完成させます。

8. 環境変数

8.1 .env.local の設定方法

Next.js では、プロジェクトルートに `.env.local` ファイルを作成して環境変数を設定します。

```
# .env.local (プロジェクトルートに作成)

# データベース接続情報 (サーバーサイドのみ)
DATABASE_URL="postgresql://user:password@localhost:5432/bookapp"

# Supabase 接続情報 (サーバーサイドのみ)
SUPABASE_URL="https://xxxxx.supabase.co"
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6I.."
```

```
# Supabase 接続情報 (クライアントサイドでも使用可能)
NEXT_PUBLIC_SUPABASE_URL="https://xxxxx.supabase.co"
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6I.."
```

重要: `.env.local` は `.gitignore` に含まれているため、Git にコミットされません。APIキーやパスワードなどの機密情報を安全に管理できます。

8.2 NEXT_PUBLIC_ プレフィックス

環境変数のアクセス範囲は、変数名のプレフィックスによって決まります。

NEXT_PUBLIC_ あり	NEXT_PUBLIC_ なし
NEXT_PUBLIC_SUPABASE_URL	SUPABASE_SERVICE_ROLE_KEY
✓ Server Component	✓ Server Component
✓ Client Component	✗ Client Component (undefined)
⚠ ブラウザの JS に含まれる	✓ ブラウザには絶対に送信されない

```
// Server Component での使用
// app/books/page.tsx

export default async function BooksPage() {
  // ✓ どちらもアクセス可能
  const url = process.env.SUPABASE_URL;
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  const publicUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;

  console.log(url);           // "https://xxxxx.supabase.co"
  console.log(serviceKey);    // "eyJhbGciOi..."
  console.log(publicUrl);     // "https://xxxxx.supabase.co"

  return <div>...</div>;
}
```

```
// Client Component での使用
// components/SomeClient.tsx
"use client";

export function SomeClient() {
  // ✓ NEXT_PUBLIC_ 付きはアクセス可能
  const publicUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
  console.log(publicUrl); // "https://xxxxx.supabase.co"

  // ✗ NEXT_PUBLIC_ なしは undefined
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;
  console.log(serviceKey); // undefined (安全!)

  return <div>...</div>;
}
```

使い分けの原則:

環境変数の種類	プレフィックス	使用例
機密情報	なし	<code>SUPABASE_SERVICE_ROLE_KEY</code> 、 <code>DATABASE_URL</code>
公開しても安全な情報	<code>NEXT_PUBLIC_</code>	<code>NEXT_PUBLIC_SUPABASE_URL</code> 、 <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>

8.3 Supabase の接続情報の管理

書籍管理アプリでは、Supabase を使います。環境変数の設定例は以下の通りです。

```
# .env.local

# Supabase プロジェクト URL (公開OK)
NEXT_PUBLIC_SUPABASE_URL="https://abcdefghijklm.supabase.co"

# Supabase 匿名キー (公開OK - Row Level Security で保護される)
NEXT_PUBLIC_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

# Supabase サービスロールキー (絶対に公開してはいけない)
SUPABASE_SERVICE_ROLE_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
// lib/supabase/client.ts
// Client Component 用の Supabase クライアント

import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}
```



```
// lib/supabase/server.ts
// Server Component 用の Supabase クライアント

import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // Server Component からの呼び出し時は
            // cookie のセットができないが、問題ない
          }
        },
      },
    },
  );
}
```

次章で詳しく解説: Supabase のセットアップと実際の接続方法は、第5章「Supabase 入門」で詳しく扱います。ここでは環境変数の管理方法だけ理解しておいてください。

9. プロジェクト構成のベストプラクティス

9.1 ディレクトリ構成例

bookshelf-app/

app app/ (ルーティング・ページ)

layout.tsx

page.tsx

globals.css

books/

page.tsx

loading.tsx

error.tsx

layout.tsx

actions.ts

new/

page.tsx

[id]/

page.tsx

edit/

page.tsx

components components/ (再利用可能なUI)

ui/ (汎用UIパーツ)

Button.tsx | Input.tsx

books/ (書籍関連)

BookCard.tsx | BookForm.tsx | SearchBar.tsx

layout/ (レイアウト関連)

Header.tsx | Footer.tsx | Sidebar.tsx

lib lib/ (ユーティリティ)

supabase/

client.ts | server.ts

utils.ts

types types/ (型定義)

book.ts | user.ts

public public/ (静的ファイル)

images/

9.2 各ディレクトリの役割

ディレクトリ	役割	含まれるもの
<code>app/</code>	ルーティングとページ	page.tsx, layout.tsx, loading.tsx, error.tsx, Server Actions
<code>components/</code>	再利用可能な UI コンポーネント	ボタン、カード、フォーム、ヘッダー等
<code>lib/</code>	ビジネスロジックとユーティリティ	DB接続、ヘルパー関数、外部API呼び出し
<code>types/</code>	TypeScript 型定義	アプリ全体で使う型 (Book, User 等)
<code>public/</code>	静的ファイル	画像、フォント、favicon 等

9.3 書籍管理アプリのファイル構成

以下は、書籍管理アプリの最終的なファイル構成の全体像です。

```
// types/book.ts
// アプリ全体で使う書籍の型定義

export type Book = {
  id: string;
  title: string;
  author: string;
  published_year: number;
  description: string | null;
  cover_image_url: string | null;
  is_favorite: boolean;
  created_at: string;
  updated_at: string;
};

// フォーム送信時に使う型 (id や日時は不要)
export type BookFormData = {
  title: string;
  author: string;
  published_year: number;
  description?: string;
};
```

```

// lib/utis.ts
// 汎用ユーティリティ関数

/**
 * 日付文字列を日本語のフォーマットに変換する
 * @example formatDate("2024-01-15") → "2024年1月15日"
 */
export function formatDate(dateString: string): string {
  const date = new Date(dateString);
  return date.toLocaleDateString("ja-JP", {
    year: "numeric",
    month: "long",
    day: "numeric",
  });
}

/**
 * クラス名を結合する (falsy な値は除外)
 * @example cn("base", isActive && "active", "extra") → "base active extra"
 */
export function cn(...classes: (string | false | undefined | null)[]): string {
  return classes.filter(Boolean).join(" ");
}

```

```
// components/ui/Button.tsx
// 汎用ボタンコンポーネント

type ButtonProps = {
  children: React.ReactNode;
  variant?: "primary" | "secondary" | "danger";
  disabled?: boolean;
  type?: "button" | "submit";
  onClick?: () => void;
};

export function Button({
  children,
  variant = "primary",
  disabled = false,
  type = "button",
  onClick,
}: ButtonProps) {
  const baseClass = "btn";
  const variantClass = `btn-${variant}`;

  return (
    <button
      type={type}
      className={`${baseClass} ${variantClass}`}
      disabled={disabled}
      onClick={onClick}
    >
      {children}
    </button>
  );
}
```

パスエイリアスの設定

`@/` というパスエイリアスを使うと、ディレクトリの深さに関係なく、プロジェクトルートからの絶対パスで import できます。Next.js のプロジェクト作成時に自動で設定されます。

```
// ❌ 相対パスでの import (ディレクトリが深いと地獄)
import { BookCard } from "../../../components/books/BookCard";
import { formatDate } from "../../../lib/utils";

// ✅ パスエイリアスでの import (常にわかりやすい)
import { BookCard } from "@components/books/BookCard";
import { formatDate } from "@lib/utils";
```

この設定は `tsconfig.json` に記述されています:

```
{
  "compilerOptions": {
    "paths": {
      "@/*": ["./*"]
    }
  }
}
```

10. よくあるエラーと対処法

エラー1: "useState" は Server Component で使用できない

Error: useState only works in Client Components. Add the "use client" directive at the top of the file to use it.

原因: Server Component（デフォルト）で `useState` や `useEffect` を使おうとしている。

対処法: ファイルの先頭に `"use client"` を追加する。

```
// ❌ エラーになる
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0); // ← ここでエラー
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}

// ✅ 修正版
"use client"; // ← これを追加

import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

エラー2: "async/await" は Client Component で使用できない

Error: async/await is not yet supported in Client Components, only Server Components.

原因: `"use client"` を付けたコンポーネントで `async` 関数としてエクスポートしている。

対処法: データ取得を Server Component に移動するか、`useEffect` を使う。

```
// ❌ エラーになる
"use client";

export default async function BooksPage() {
  const books = await fetch("/api/books"); // ← async は Client Component で不可
  return <div>...</div>;
}

// ✅ 修正版 1: Server Component にする ("use client" を削除)
export default async function BooksPage() {
  const books = await fetch("https://api.example.com/books");
  return <div>...</div>;
}

// ✅ 修正版 2: Client Component のまま useEffect を使う
"use client";

import { useState, useEffect } from "react";

export default function BooksPage() {
  const [books, setBooks] = useState([]);

  useEffect(() => {
    fetch("/api/books")
      .then((res) => res.json())
      .then(setBooks);
  }, []);

  return <div>...</div>;
}
```

エラー3: "next/router" と "next/navigation" の混同

```
Error: NextRouter was not mounted.
```

原因: App Router で `next/router` を import している。App Router では `next/navigation` を使う。

```
// ❌ App Router では使えない
import { useRouter } from "next/router";

// ✅ App Router ではこちらを使う
import { useRouter } from "next/navigation";
```

エラー4: metadata と "use client" の共存

Error: You are attempting to export "metadata" from a component marked with "use client", which is unsupported.

原因: `"use client"` のファイルで `metadata` をエクスポートしている。メタデータは Server Component でのみエクスポート可能。

```
// ❌ エラーになる
"use client";

export const metadata = { title: "書籍一覧" }; // ← Client Component では不可

// ✅ 修正版: metadata は Server Component (page.tsx や layout.tsx) に置く
// app/books/page.tsx (Server Component)
export const metadata = { title: "書籍一覧" };

export default function BooksPage() {
  return <div>...</div>;
}
```

エラー5: 環境変数が undefined になる

原因: Client Component で `NEXT_PUBLIC_` プレフィックスのない環境変数を使っている。

```
// ❌ Client Component では undefined になる
"use client";

const apiKey = process.env.API_SECRET_KEY; // undefined

// ✅ NEXT_PUBLIC_ を付ける (ただし機密情報には使わない!)
const publicUrl = process.env.NEXT_PUBLIC_API_URL; // 値が取得できる
```

注意: 機密情報に `NEXT_PUBLIC_` を付けてはいけません。Server Action や API Route 経由でアクセスしてください。

エラー6: "Text content does not match server-rendered HTML" (Hydration エラー)

Warning: Text content did not match. Server: "2024年3月15日" Client: "3/15/2024"

原因: サーバーとクライアントで異なる出力が生成されている。日時のフォーマットやランダム値でよく発生する。


```
// ❌ サーバーとクライアントで結果が異なる可能性がある
export default function DateDisplay() {
  return <p>{new Date().toLocaleString()}</p>;
}

// ✅ 修正案 1: suppressHydrationWarning を使う
export default function DateDisplay() {
  return <p suppressHydrationWarning>{new Date().toLocaleString()}</p>;
}

// ✅ 修正案 2: Client Component にして useEffect で設定
"use client";

import { useState, useEffect } from "react";

export default function DateDisplay() {
  const [dateStr, setDateStr] = useState("");

  useEffect(() => {
    setDateStr(new Date().toLocaleString("ja-JP"));
  }, []);

  return <p>{dateStr}</p>;
}
```

エラー7: fetch でのキャッシュに関する警告

Warning: fetch for "https://..." on "/" used "no-store" and should not be called inside a layout or template.

対処法: `layout.tsx` 内では `cache: "no-store"` を避け、`page.tsx` で使う。

エラーの調査方法（一般的なアドバイス）

1. ターミナルのエラーメッセージを読む: Next.js のエラーメッセージは非常に詳しく、多くの場合、解決策まで提示してくれます。
2. ブラウザのコンソールを確認する: 開発者ツール (F12) の Console タブにもエラーが表示されます。
3. Server Component と Client Component の境界を確認する: 多くのエラーは、この2つの境界での誤りが原因です。
4. Next.js の公式ドキュメントを参照する: <https://nextjs.org/docs> には詳細な説明とトラブルシューティングがあります。

まとめ

この章では Next.js の基礎を広く学びました。

トピック	学んだこと
Next.js とは	React のフルスタックフレームワーク。SSR/SSG/CSR に対応
App Router	ファイルベースルーティング。page.tsx, layout.tsx 等の特殊ファイル
Server/Client Components	デフォルトは Server。インタラクティブ性が必要なら "use client"
レイアウト	ルートレイアウトとネストされたレイアウトで共通UIを効率的に管理
ナビゲーション	Link コンポーネント（宣言的）と useRouter フック（命令的）
データフェッチ	Server Component なら async/await で直接。Client は useEffect
Server Actions	"use server" でサーバー処理を直接呼び出し。フォーム処理に最適
環境変数	NEXT_PUBLIC_ の有無でアクセス範囲が変わる
プロジェクト構成	app/, components/, lib/, types/ の4層構造

次の章では: Supabase のセットアップを行い、実際のデータベースと接続します。書籍管理アプリのバックエンドを構築し、この章で学んだ Server Components や Server Actions を使ってデータの読み書きを実装していきます。

確認問題

以下の問いに答えて、理解度を確認してみましょう。

1. Server Component と Client Component の違い を3つ挙げてください。
2. 以下のコンポーネントは Server / Client のどちらにすべきですか？ - データベースから書籍一覧を取得して表示するコンポーネント - 検索バー（ユーザーの入力をリアルタイムに反映） - 書籍カードの静的な表示（クリックイベントなし）
3. NEXT_PUBLIC_API_KEY と API_SECRET_KEY はそれぞれどこで使用できますか？
4. layout.tsx と page.tsx の違いは何ですか？
5. Server Actions を使う利点を2つ挙げてください。

▶ 解答を見る